

Софийски университет "Св. Кл. Охридски"

Факултет по математика и информатика

Курсов Проект

на тема: "Система за извличане на информация от алгоритмични задачи на български език"

Студент: **Теодор Светославов Костадинов** Ф.Н. 5MI3400816

Курс: "Извличане на Информация", Учебна година: 2025/26

Преподаватели: **проф. Иван Койчев**

Декларация за липса плагиатство:

- Плагиатство е да използваш, идеи, мнение или работа на друг, като претендираш, че са твои. Това е форма на преписване.
- Тази курсова работа е моя, като всички изречения, илюстрации и програми от други хора са изрично цитирани.
- Тази курсова работа или нейна версия не са представени в друг университет или друга учебна институция.
- Разбирам, че ако се установи плагиатство в работата ми ще получа оценка "Слаб".

13.2.26 г. Подпис на студента:

Съдържание

1. Увод
2. Подходи за извличане на информация от образователни ресурси
    - Съществуващи решения
    - Методи за извличане на информация
3. Проектиране
    - 3.1. Данни
    - 3.2. Предобработка на текста
    - 3.3. Модели за извличане на информация
    - 3.4. Допълнителни NLP експерименти
    - 3.5. Методика на оценка
    - 3.6. Уеб интерфейс
4. Реализация, тестване и експерименти
    - 4.1. Използвани технологии, платформи и библиотеки
    - 4.2. Провеждане на експерименти и резултати
5. Заключение
6. Използвана литература

---

1. Увод

Всяка година от 2018 г. насам в курса по Структури от данни и алгоритми се добавят около 30 нови задачи в платформата *HackerRank*. С нарастването на броя задачи до над 650, подготовката на студентите и намирането на подобни задачи за упражнение става все по-трудно. Липсва единна система за търсене, филтриране и препоръчване на задачи по тема и сложност.

Целта на проекта е изграждане на система за извличане на информация, която:

1. **Извлича** условията на алгоритмични задачи от *HackerRank* чрез уеб скрейпинг
2. **Индексира** задачите чрез множество модели за извличане, *TF-IDF*, *BERT* и хибриден подход
3. **Търси и подрежда** задачи по прилика спрямо входна заявка, оценени чрез стандартни IR метрики
4. **Предоставя** уеб интерфейс за търсене, преглед и препоръчване на задачи за студенти

---

2. Подходи за извличане на информация от образователни ресурси

Съществуващи решения

Извличането на информация от образователни ресурси е активна изследователска област. Системи за търсене на подобни задачи съществуват в платформи като *LeetCode* и *Codeforces*, но използват ръчно зададени тагове, а не автоматично извличане на информация от текста на условията. За български език подобна система не е реализирана.

Методи за извличане на информация

Съгласно учебника на Manning, Raghavan и Schütze [1], основните подходи за извличане на информация включват:

- **Векторно-пространствен модел с *TF-IDF* тегла**, документите и заявките се представят като вектори в пространство от термини, а приликата се измерва чрез косинусова мярка
- **Езикови модели**, вероятностно моделиране на текст, в съвременния контекст, предварително обучени *Transformer* модели
- **Оценка на системите за извличане на информация**, стандартни метрики като *Precision@K*, *MRR*, *MAP*
- **Обработка на текста**, токенизация, лематизация, нормализация, премахване на стоп-думи
- **Обхождане и индексиране на уеб**, автоматично извличане на данни от уеб платформи

В настоящия проект са приложени и трите подхода, като допълнително е реализиран хибриден модел, комбиниращ *TF-IDF* и *BERT*. Извършени са и допълнителни експерименти с NLP подходи за класификация, включително PMI-базиран класификатор и извличане на семантични слотове.

---

3. Проектиране

3.1. Данни

Уеб скрейпинг

Данните са извлечени от платформата *HackerRank* чрез автоматизиран скрейпър, реализиран с библиотеката *Playwright*. Достъпът до задачите изисква автентикация в платформата, тъй като условията са достъпни само за логнати потребители. Скрейпърът обхожда списъка от състезания и за всяка задача извлича:

- Условие на задачата
- Метаданни: трудност, максимален брой точки, успеваемост, тагове, състезание

Скрейпърът поддържа динамично разширяване на корпуса, при добавяне на нови състезания се скрейпват само новите задачи без повторно обхождане на вече извлечените. Реализирана е и дедупликация, задачи които се срещат в повече от едно състезание не се записват повторно.

Изходният корпус съдържа **658 задачи** с текстови файлове и JSON метаданни.

Анотиране

Анотирането е извършено чрез слабо анотиране с *OpenAI* агент. Класовете задачи са дефинирани според категориите от конспекта на курса по СДА. Получени са **329 анотирани задачи** с 14 категории: *binary_search_tree*, *linked_list*, *graph_traversal*, *shortest_path*, *sorting*, *dynamic_programming*, *stack_queue*, *priority_queue*, *hash_table*, *tree*, *minimum_spanning_tree*, *searching*, *string* и *eulerian_hamiltonian*.

3.2. Предобработка на текста

Приложени са стандартни техники за обработка на текст:

- **Нормализация**, премахване на HTML тагове, *markdown* форматиране, *LaTeX* формули, привеждане към малки букви
- **Извличане на секции**, от суровия текст се извличат само значимите части: условие, входен формат и изходен формат
- **Токенизация и лематизация**, чрез библиотеката *CLASSLA*, специализирана за южнославянски езици [4]
- **Премахване на стоп-думи**, използване на *stopwordsiso* за български език

3.3. Модели за извличане на информация

Модел I: *TF-IDF* векторно-пространствен модел. Лематизираният текст е представен като *TF-IDF* вектори с униграми и биграми. Приликата между документи се измерва чрез косинусова мярка. Параметри на модела: min_df=1, max_df=0.95, ngram_range=(1,2).

Модел II: *BERT* езиков модел. Използван е предварително обучен многоезичен *Sentence Transformer* модел (`paraphrase-multilingual-MiniLM-L12-v2`) [3], който генерира плътни вектори с 384 измерения за всяка задача. За *BERT* се прилага облекчена предобработка, без лематизация, тъй като моделът има собствен токенизатор. Приликата се измерва отново чрез косинусова мярка.

Модел III: Хибриден модел. Хибридният модел комбинира резултатите от *TF-IDF* и *BERT* чрез линейна интерполация:

```
score = (1 - α) × tfidf_similarity + α × bert_similarity
```

Параметърът α е определен чрез *sweep* от 0.0 до 1.0 със стъпка 0.1. Оптималната стойност е **α = 0.4**, 60% *TF-IDF* + 40% *BERT*.

3.4. Допълнителни NLP експерименти

PMI-базиран класификатор. За всяка категория задачи се индуцират до 30 правила с най-висок PMI *score* на базата на леми, биграми и глаголи, извлечени чрез *CLASSLA*. Допълнителен вариант със слети класове обединява сходни категории.

Извличане на семантични слотове. Ръчно дефинирани *slot* правила с лематизирани *cue* думи за български извличат структурирана информация за всяка задача, *topic* и *paradigm*.

3.5. Методика на оценка

Оценката е извършена чрез *leave-one-out* метод: за всяка от 329-те задачи се търсят Top-5 най-подобни сред останалите, като се проверява дали принадлежат към същата категория. Използвани метрики:

- **P@K**, дял на релевантните резултати сред Top-K
- **MRR**, средна реципрочна позиция на първия релевантен резултат
- **MAP@5**, средна точност на подреждането до позиция 5

3.6. Уеб интерфейс

Реализиран е интерактивен уеб интерфейс чрез *Streamlit* [5] с четири раздела:

- **Search**, търсене по свободен текст с хибриден модел и настройка на α
- **Browse**, преглед на всички задачи в сортируема таблица с филтри по тема, трудност, състезание и успеваемост
- **Topics**, статистика и визуализация по теми, разпределение на трудността, успеваемост, списък на задачите от лесни към трудни
- **Practice**, препоръчване на следваща задача на базата на решени задачи и намиране на най-подобна нерешена задача чрез предварително изчислена матрица на приликите

---

4. Реализация, тестване и експерименти

4.1. Използвани технологии, платформи и библиотеки

Таблица 1. Използвани технологии

Компонент	**Технология**
Уеб скрейпинг	*Playwright*
TF-IDF индексиране	*scikit-learn* (*TfidfVectorizer*)
BERT embeddings	*sentence-transformers*
NLP за български	*CLASSLA*
Стоп-думи	*stopwordsiso*
Уеб интерфейс	*Streamlit*
Анотиране	*OpenAI API*

Езикът за програмиране е *Python*. *Playwright* е избран за уеб скрейпинг поради нуждата от аутентикация. *CLASSLA* е единственият наличен NLP *pipeline* с поддръжка на лематизация за български език. *Sentence Transformers* предоставя предварително обучени многоезични модели, подходящи за генериране на плътни вектори от текст на български.

Таблица 2. Структура на проекта

Файл	**Описание**
`scraper.py`	Уеб скрейпър за извличане на задачи от HackerRank
`classifier.py`	Анотиране на задачи чрез OpenAI API
`similarity_search.py`	Базов TF-IDF модел без лематизация
`similarity_search_nlp.py`	TF-IDF модел с CLASSLA лематизация и оценка
`similarity_search_bert.py`	BERT модел с оценка
`similarity_search_hybrid.py`	Хибриден модел с weight sweep и оценка
`search_app.py`	Streamlit уеб интерфейс
`rule_based_topic_classifier.py`	PMI-базиран класификатор
`intent_extraction.py`	Извличане на семантични слотове

4.2. Провеждане на експерименти и резултати

Допълнителни NLP експерименти

Таблица 3. Резултати от NLP експериментите за класификация

Експеримент	**Метрика**	**Стойност**
PMI класификатор	Accuracy	0.333
PMI със слети класове	Top-1	0.226
PMI със слети класове	Top-5	0.903
Извличане на слотове	Accuracy	0.307

PMI класификаторът постига *Accuracy* = 0.333, като категории с характерна лексика като *binary_search_tree* и *linked_list* се разпознават с F1 = 1.0, докато абстрактни категории като *dynamic_programming* остават с F1 = 0.0. Вариантът със слети класове показва, че правилната категория попада в Top-5 в 90.3% от случаите. Извличането на семантични слотове постига *Accuracy* = 0.307, *macro* F1 = 0.186.

Rule-based подходите са недостатъчни за класификация на алгоритмични задачи, което мотивира използването на векторни модели за извличане на информация.

Сравнение на моделите за извличане

Таблица 4. Сравнение на моделите за извличане на информация

Модел	**P@1**	**P@3**	**P@5**	**MRR**	**MAP@5**
TF-IDF (лематизиран)	0.511	0.441	0.423	0.614	0.587
BERT (multilingual)	0.465	0.424	0.403	0.590	0.560
Хибриден (α=0.4)	**0.535**	**0.453**	**0.423**	**0.638**	**0.605**

Хибридният модел постига най-добри резултати по всички метрики, с P@1 = 53.5% и MRR = 0.638.

Таблица 5. Анализ по категории, P@1

Категория	**TF-IDF**	**BERT**	**Хибриден**	**Брой**
binary_search_tree	0.842	0.763	**0.921**	38
linked_list	**0.840**	0.800	**0.840**	25
graph_traversal	**0.673**	0.596	0.654	52
shortest_path	**0.600**	0.400	**0.650**	20
sorting	**0.500**	0.341	0.477	44
dynamic_programming	0.368	0.353	**0.441**	68
stack_queue	**0.348**	0.261	0.304	23
hash_table	0.219	**0.281**	0.219	32
minimum_spanning_tree	0.250	**0.625**	0.375	8

Анализ на резултатите

TF-IDF превъзходство при лексикално богати категории. Категории с характерна терминология като *binary_search_tree* и *linked_list* се разпознават с P@1 над 84% от *TF-IDF*. Лексикалното съвпадение е силно, когато задачите споделят специфични ключови думи.

BERT предимство при семантично близки задачи. *BERT* постига по-добри резултати при *hash_table* и *minimum_spanning_tree*, където семантичното разбиране помага за разграничаване на задачи с различна лексика но сходно значение.

Хибридният модел комбинира предимствата. При α=0.4 хибридният модел подобрява P@1 спрямо *TF-IDF* при 4 от 9 категории, най-значимо при *binary_search_tree* (0.842 → 0.921) и *dynamic_programming* (0.368 → 0.441).

Трудни категории. *dynamic_programming* и *hash_table* остават най-трудни за всички модели, тъй като се определят от метода на решение, а не от лексиката на условието.

---

5. Заключение

Обобщение

Проектът реализира пълна система за извличане на информация от алгоритмични задачи на български език, включваща:

- Уеб скрейпър за автоматично извличане на 658 задачи от *HackerRank* с поддръжка на автентикация, динамично разширяване и дедупликация
- Три модела за търсене по прилика: *TF-IDF*, *BERT* и хибриден

- Допълнителни NLP експерименти с PMI класификатор и извличане на семантични слотове
- Стандартна IR оценка с *P@K*, *MRR* и *MAP* метрики
- Интерактивен уеб интерфейс с търсене, преглед, статистика по теми и препоръчване на задачи

Хибридният модел, 60% *TF-IDF* + 40% *BERT*, постига най-добрия P@1 = 53.5% и MRR = 0.638.

Бъдещо развитие

- **Разширяване на заявки**, автоматично обогатяване на заявката с ключови фрази от анотациите
- ***Learning to Rank***, обучение на модел за оптимално комбиниране на множество сигнали
- **Подобряване на анотациите**, ръчна проверка на *OpenAI* етикетите за по-надеждна оценка
- **Публично разгръщане**, предварително изчислени индекси и *deploy* в *Streamlit Cloud* за достъп на всички студенти

---

6. Използвана литература

[1] Manning, Christopher D., Prabhakar Raghavan, and Hinrich Schütze. *Introduction to Information Retrieval.* Cambridge University Press, 2008. Web. http://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.120.6095&rep=rep1&type=pdf

[2] Baeza-Yates, Ricardo, and Berthier Ribeiro-Neto. *Modern Information Retrieval.* Addison-Wesley, 1999. Print.

[3] Reimers, Nils, and Iryna Gurevych. "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks." *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing.* 2019. Print.

[4] Ljubešić, Nikola, and Kaja Dobrovoljc. "What Does Neural Bring? Analysing Improvements in Morphosyntactic Annotation and Lemmatisation of Slovenian, Croatian and Serbian." *Proceedings of the 7th Workshop on Balto-Slavic Natural Language Processing.* 2019. Web. https://github.com/clarinsi/classla

[5] *Streamlit: A Faster Way to Build and Share Data Apps.* Streamlit Inc., 2024. Web. https://streamlit.io